KNOTOPIAN INSTITUTE OF TECHNOLOGY · DEPARTMENT OF CSE

CS304 Database Management Systems

Lesson notes, Units 1 to 3 · B.Tech CSE, Semester 3 (Class CSE-2A)

Prepared by Mr. R. Ravi Kumar, Assistant Professor ("Ravi sir"), from the classes held in the odd semester 2026–27

Classes: Monday Period 2; Thursday Periods 1 and 3

Version 2026.1 · Shared through the college system

Syllabus for this semester

Unit	Topics	In this semester's assessments?
Unit 1	Data models, ER model, relational model, keys, relational algebra	Yes
Unit 2	SQL: queries, joins, subqueries, aggregates, views, constraints	Yes
Unit 3	Functional dependencies and normalisation (1NF to BCNF); transactions and concurrency control	Yes
Unit 4	Query processing, indexing and NoSQL databases	**No.** Unit 4 is not part of this semester's syllabus. It has moved to CS402 Advanced Database Systems in Semester 4. Do not prepare Unit 4 for the Internal Assessment or the End Semester Examination of CS304.

Internal Assessment pattern (CS304)

The CS304 Internal Assessment covers Units 1, 2 and 3. It is a two-hour written test for 50 marks:

- Part A: ten questions of 2 marks each (all compulsory).
- Part B: answer any two of three questions, 15 marks each. One Part B question will always be a normalisation problem using an example from our classes.

The date of the Internal Assessment is in the class calendar in the college system. If the date changes, the new date will be announced by circular; always check the college system for the latest date.

Unit 1 · Data models and the relational model

1.1 Why databases

Before databases, every office of the college kept its own files: the library had its own student list, the hostel had another, and the placement cell had a third. When a student changed their phone number, three files had to be corrected, and usually one was missed. A database keeps one copy of the data, shared by everyone who needs it, with rules that keep it correct.

1.2 The ER model: our library example

In class we drew the ER diagram of the KIT Central Library. The entities are **Student** (RollNo, Name, Department, Year) and **Book** (BookID, Title, Author, Shelf). The relationship **Issues** connects them and has the attributes IssueDate and DueDate. A student can borrow many books and a book can be borrowed by many students over time, so Issues is a many-to-many relationship.

Sir pointed out a common mistake: students put DueDate inside Book. That is wrong, because the due date belongs to one particular issue of the book, not to the book itself.

1.3 Keys

A **superkey** is any set of attributes that identifies a row uniquely. A **candidate key** is a minimal superkey: if you remove any attribute from it, it stops being a key. The **primary key** is the candidate key we choose.

Class example: in Student(RollNo, AadhaarNo, Email, Name), RollNo, AadhaarNo and Email are each candidate keys. (RollNo, Name) is a superkey but not a candidate key, because RollNo alone is already a key. Sir said: "If you can remove something and it still works, it was never minimal."

A **foreign key** is an attribute that refers to the primary key of another table, for example RollNo in the Issues table refers to Student.

1.4 Relational algebra on the placement database

We used a small version of the placement cell's database:

```
Drive(DriveID, Company, Category, MinCGPA)
Registration(RollNo, DriveID)
Student(RollNo, Name, Year, CGPA, Arrears)
```

Selection (σ) picks rows: $\sigma_{\text{Category='dream'}}(\text{Drive})$ gives the dream drives. Projection (π) picks columns: $\pi_{\text{Company}}(\text{Drive})$ gives the company names. A join (⋈) combines tables: Registration ⋈ Student gives the details of everyone who registered.

Exercise from class: "Names of students registered for Zoho" = $\pi_{\text{Name}}(\sigma_{\text{Company='Zoho'}}(\text{Drive}) \bowtie \text{Registration} \bowtie \text{Student})$.

Unit 2 · SQL

2.1 Basic queries

```
SELECT Name, CGPA
FROM Student
WHERE Year = 4 AND Arrears = 0
ORDER BY CGPA DESC;
```

Remember: WHERE filters rows before grouping; HAVING filters groups after grouping.

2.2 Sir's favourite query: who is eligible for a drive

This query came up in almost every class. A student is eligible for the Zoho drive if they are in the final year, have a CGPA of at least 7.0 and have no standing arrears:

```
SELECT s.RollNo, s.Name
FROM Student s
WHERE s.Year = 4
  AND s.CGPA >= 7.0
  AND s.Arrears = 0;
```

Sir's warning: "The database only stores the data. Whether a student is allowed to register is a rule of the college. Write the rule in your query; don't assume the table will stop you."

2.3 Joins

An INNER JOIN returns only matching rows. A LEFT JOIN returns every row of the left table, with NULLs where there is no match. Class example: list every final-year student and the drives they registered for, including students who registered for none:

```
SELECT s.Name, d.Company
FROM Student s
LEFT JOIN Registration r ON r.RollNo = s.RollNo
LEFT JOIN Drive d ON d.DriveID = r.DriveID
WHERE s.Year = 4;
```

2.4 Aggregates and GROUP BY

```
SELECT d.Company, COUNT(*) AS registrations
FROM Registration r JOIN Drive d ON d.DriveID = r.DriveID
GROUP BY d.Company
HAVING COUNT(*) > 10;
```

2.5 Subqueries

Students with a CGPA above the class average:

```
SELECT Name FROM Student
WHERE CGPA > (SELECT AVG(CGPA) FROM Student);
```

2.6 Views and constraints

A view is a saved query: CREATE VIEW FinalYear AS SELECT * FROM Student WHERE Year = 4. Constraints keep data correct: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK. Class example: CHECK (CGPA BETWEEN 0 AND 10). In the Registration table, UNIQUE (RollNo, DriveID) stops a student registering twice for the same drive.

Unit 3 · Normalisation, transactions and concurrency

3.1 Functional dependencies

A functional dependency X → Y means: if two rows agree on X, they must agree on Y. Example: RollNo → Name. A dependency is **partial** if Y depends on only part of a composite key, and **transitive** if X → Y holds only through another attribute (X → Z and Z → Y).

3.2 First normal form (1NF): the hostel room example

Our example was the hostel room allotment sheet, where the warden had written two or three phone numbers in one cell: Room(RoomNo, Block, StudentPhones). A table is in 1NF only if every cell holds a single value. Fix: one row per student per room, RoomAllotment(RoomNo, Block, RollNo, Phone).

3.3 Second normal form (2NF): the library issue register

Our 2NF example was the library issue register: Issue(RollNo, BookID, IssueDate, BookTitle), with the key (RollNo, BookID). BookTitle depends only on BookID, which is part of the key. That is a partial dependency, so the table is not in 2NF. Fix: move BookTitle into Book(BookID, BookTitle) and keep Issue(RollNo, BookID, IssueDate).

3.4 Third normal form (3NF): the canteen order slip

For 3NF, sir used the KIT canteen order slip. Each slip has an order number, the student's roll number, the counter where the food was collected and the name of the person in charge of that counter:

```
Order(OrderNo, RollNo, CounterNo, CounterIncharge)

OrderNo   -> CounterNo
CounterNo -> CounterIncharge
```

CounterIncharge depends on OrderNo only through CounterNo. That is a transitive dependency, so the table is not in 3NF. When the in-charge of Counter 3 changes, every old slip for Counter 3 would have to be updated. Fix:

```
Order(OrderNo, RollNo, CounterNo)
Counter(CounterNo, CounterIncharge)
```

Sir said this canteen example will appear in some form in the Internal Assessment, so learn to explain why the original table is not in 3NF, not just how to split it.

3.5 Boyce-Codd normal form (BCNF): lab slot booking

Our BCNF example was booking laboratory slots: Booking(Lab, Slot, Faculty), where each faculty member always uses one lab (Faculty → Lab), and (Lab, Slot) → Faculty. Faculty is not a key, yet it determines Lab, so the table is in 3NF but not in BCNF. Fix: FacultyLab(Faculty, Lab) and Booking(Faculty, Slot).

3.6 Transactions and ACID

A transaction is a group of operations that must succeed or fail together. ACID stands for Atomicity, Consistency, Isolation and Durability. Class example: moving a student from one placement drive to another must remove the old registration and add the new one together; doing only one of them leaves the data wrong.

3.7 Concurrency: the last seat in the Zoho drive

Sir's example for concurrency was the last seat in the Zoho placement drive. Two students click "register" at the same moment. Both read SeatsLeft = 1, both register and both reduce SeatsLeft, so the drive ends up with -1 seats. This is the lost update problem.

The fix is to lock the drive's row while one registration is being processed:

```
BEGIN;
SELECT SeatsLeft FROM Drive WHERE DriveID = 7 FOR UPDATE;
-- if SeatsLeft > 0:
INSERT INTO Registration (RollNo, DriveID) VALUES ('22CS041', 7);
UPDATE Drive SET SeatsLeft = SeatsLeft - 1 WHERE DriveID = 7;
COMMIT;
```

The second student waits until the first transaction commits, then reads SeatsLeft = 0 and is refused.

3.8 Schedules, serialisability and two-phase locking

A schedule is the order in which the operations of several transactions run. A schedule is conflict serialisable if it can be rearranged, by swapping non-conflicting operations, into some serial order. Two-phase locking (2PL) guarantees conflict serialisability: a transaction takes all its locks (growing phase) before it releases any (shrinking phase).

3.9 Deadlock: two teachers approving requests

Sir's deadlock example: Teacher 1 locks request R1 and then wants R2; at the same time Teacher 2 locks R2 and then wants R1. Each waits for the other forever. Databases detect this with a wait-for graph and roll back one of the transactions.

Common mistakes in last year's answers

- Calling a superkey a candidate key without checking that it is minimal (section 1.3).
- Splitting a table for 3NF without stating the transitive dependency. In the canteen example you must write OrderNo → CounterNo → CounterIncharge.
- Writing HAVING instead of WHERE for conditions on single rows.
- Answering Unit 4 questions (indexing, NoSQL) from internet notes. Unit 4 is not in this semester's syllabus.
- Explaining concurrency with a bank example when the question asks for "the example discussed in class", which is the Zoho last-seat example.

These notes are for KIT students of CSE-2A. Where these notes differ from a textbook, follow these notes for CS304 assessments.